

김범진

👤 총 경력 5년 6개월 📞 010-8849-8539 ✉️ hypnos709@gmail.com 🌐 [GitHub](#) 👤 [LinkedIn](#)

문제 인식부터 개선, 결과 검증까지의 제품 개선 사례

목차

1. 이중결제 이슈를 해소하기 위해 결제 흐름을 서버 중심으로 재설계
2. Sendbird 의존 채팅을 자체 채팅 서버로 내재화
3. 앱 전환 중심 구조를 웹 완결형 사용자 경험으로 재구성
4. 표준 개발 프로세스를 정립하고 AI 에이전트가 재현할 수 있는 개발 하네스를 구축
5. 서로 다른 데이터 모델을 일관된 인터랙티브 차트 경험으로 통합

Case Studies

이중결제 이슈를 해소하기 위해 결제 흐름을 서버 중심으로 재설계

서버사이드 아키텍처 전환 · 동일 이슈 재발 방지

문제 인식

결제 도중 이탈이나 중복 요청이 발생하면 이중결제와 결제 완료/주문 미기록 상태가 함께 생길 위험이 있었습니다. 주문 금액을 클라이언트 요청에 의존하는 구간이 남아 있어 금액 위변조 가능성이 있었고, 유료와 무료 주문의 처리 방식이 달라 주문/결제 데이터 정합성도 흔들릴 수 있었습니다.

분석

스케줄러로 사후 환불을 보정하는 방식만으로는 결제와 주문 사이 불일치를 근본적으로 해결하기 어렵다고 판단했습니다. 금액 검증과 주문 상태 변경은 신뢰할 수 있는 서버에서만 확정되어야 했고, PG confirm은 네트워크 지연과 타임아웃이 생길 수 있어 즉시 실패 처리 대신 진행 중 상태를 추적할 수 있는 구조가 필요했습니다.

실행

- 주문 생성과 결제 확정의 책임을 서버가 갖는 구조로 결제 플로우를 다시 설계했습니다.
- 주문 생성 단계에서 서버가 계산한 최종 금액과 클라이언트 요청 금액을 먼저 검증하고, 이후 Redis 세션의 가격 정보로 한 번 더 검증했습니다.
- 서버에 `WAITING_PAYMENT` 상태를 도입해 결제 시도 단위의 주문번호를 먼저 고정하고, 재시도·재진입에도 동일 주문으로 처리되도록 멍등성을 보장했습니다.
- Bootpay confirm 타임아웃 시 `PROCESSING` 응답을 반환하고, 주문 상태 조회 API로 폴링할 수 있는 흐름을 추가했습니다.
- CARD 결제와 FREE 주문을 분기 처리해 무료 주문은 Bootpay 검증 없이 완료하고, 무료 주문에서도 정합성이 깨지지 않도록 로직을 수정했습니다.

결과

- 반복적으로 발생하던 중복 결제 이슈를 해결했습니다.
- 서버사이드 결제 구조 릴리즈 이후 동일 이슈가 재발하지 않았습니다.
- 주문 생성, 결제 확정, 환불 정책의 책임을 서버로 일원화해 운영자가 같은 주문번호 기준으로 결제 상태를 추적할 수 있게 했습니다.

Before

1 Client → PG Provider: 결제창 호출

2 PG Provider: 카드 승인 완료

3 PG Provider → Client: onConfirm

4 Client → PG Provider: 즉시 결제 확정

5 PG Provider → Client: receiptId 전달

6 Client → Backend: 검증 및 DB 저장 요청

7 이 구간에서 실패하면 결제 완료 / 주문 미기록 상태가 남음

After

- 1 Client → Backend: 주문 생성 API 호출
- 2 Backend → Client: orderId / paymentId 반환
- 3 Client → PG Provider: 결제창 호출
- 4 PG Provider → Client: receiptId 전달
- 5 Client → Backend: 결제 확정 API 호출
- 6 Backend: 세션 검증, receipt 검증, 결제 상태 확정
- 7 Backend → Client: COMPLETED 또는 PROCESSING 응답
- 8 Client → Backend: 필요 시 상태 조회 API로 폴링
- 9 최종 주문 완료 상태를 서버 기준으로 확정

Sendbird 의존 채팅을 자체 채팅 서버로 내재화

자체 채팅 서버 구축 · 월 \$300 SaaS 비용 절감 · Phase 2 월 \$1,700 절감 예상

문토와 데이팅 서비스의 채팅은 Sendbird에 의존하고 있었고, MAU 기반 비용과 정책 커스터마이징 제약이 함께 커지고 있었습니다. 자체 채팅 서버를 구축해 500명 규모 트래픽을 분산했고, 1차 전환으로 월 약 300달러의 SaaS 비용을 절감했습니다.

문제 인식

외부 채팅 SaaS는 빠른 도입에는 유리했지만, 사용자 규모가 커질수록 비용 부담이 커졌고 푸시, 금칙어, 제재, 백오피스 모니터링 같은 내부 운영 정책을 세밀하게 반영하기 어려웠습니다.

분석

단순 SDK 교체가 아니라 비용, 운영 권한, 메시지 정합성을 함께 다루는 서버 전환이 필요하다고 판단했습니다. REST 요청과 실시간 연결은 성격이 달라 분리하고, 메시지 순서와 누락 복구는 서버가 발급한 messageId를 기준으로 맞췄습니다.

실행

- REST 서버와 WebSocket 서버를 분리해 요청 처리와 실시간 연결 책임을 나눴습니다.
- PostgreSQL에 메시지를 저장하고 Redis Pub/Sub으로 WebSocket 다중 노드를 동기화했습니다.
- messageId 기준 정렬, 재연결 gap recovery, 읽음 커서 기반 unread count로 메시지 정합성을 맞췄습니다.
- 1차 500명 규모 트래픽을 자체 서버로 분산해 월 약 300달러의 SaaS 비용을 절감했습니다.

결과

- Sendbird 의존도를 낮추고 채팅 운영 정책을 내부에서 통제할 수 있는 기반을 만들었습니다.
- 1차 월 약 \$300 절감, Phase 2 월 약 \$1,700 추가 절감이 가능한 비용 절감 경로를 만들었습니다.
- 메시지 저장, 읽음 처리, 푸시, 백오피스 모니터링을 서비스 정책에 맞게 확장할 수 있게 했습니다.

Before

- 1 앱 UI가 Sendbird SDK를 통해 외부 채팅 SaaS에 직접 의존
- 2 MAU 7만 요금제 기준 월 \$4,300 수준 비용 발생
- 3 푸시, 금칙어, 제재, 백오피스 모니터링 정책을 내부 요구에 맞추기 어려움
- 4 메시지 저장, 읽음 처리, 운영 모니터링의 통제권이 외부 솔루션 경계에 묶임

After

문토 앱

Flutter SDK

Closer 앱

TypeScript SDK

Socket.IO + REST + JWT

REST 서버

채팅방 · 히스토리 · 백오피스

WebSocket 서버

메시지 · 읽음 · 상태

PostgreSQL

메시지 영구 저장

Redis Pub/Sub

다중 노드 동기화

Kafka

제재 · 자동신고 이벤트

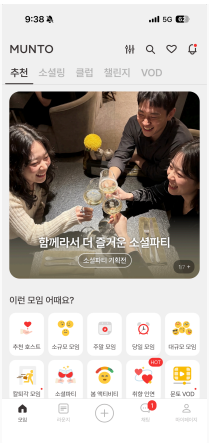
chat-notification

FCM/APNs 푸시

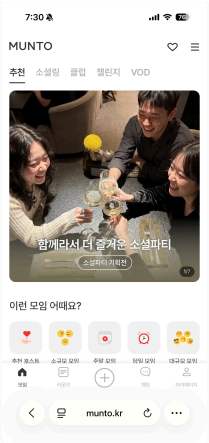
1차 월 \$300 절감 · Phase 2 월 \$1,700 추가 절감 예상

앱 전환 중심 구조를 웹 완결형 사용자 경험으로 재구성

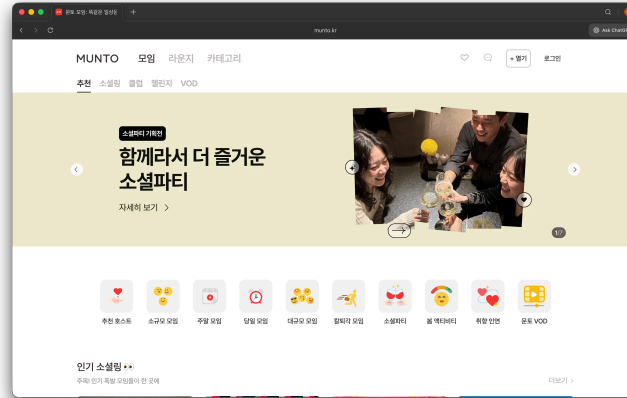
웹 확장 · Web MAU 8만 → 12만 · 전체 MAU 25만+



기존 앱 메인화면



신규 웹 모바일 메인화면



신규 웹 데스크톱 메인화면

문제 인식

앱 소개용 링크 페이지로는 웹 유입을 실제 사용 전환으로 연결하기 어려웠습니다. 웹에서도 로그인, 상태관리, 결제까지 이어지는 완결된 경험이 필요했습니다. 기존 웹은 소개 페이지 중심이어서 실제 서비스 화면에서 필요한 회원 상태, 프로필, 인증 토큰 같은 상태를 다루는 구조가 없었습니다.

분석

단순히 랜딩 페이지를 개선하는 것만으로는 전환 문제를 해결하기 어렵다고 판단했습니다. 웹만으로 서비스를 이용하고 싶은 수요가 이미 있었고, 실제 전환을 만들려면 웹도 앱처럼 로그인 이후 탐색과 결제까지 이어지는 완결된 구조를 가져야 했습니다. 웹 유입 사용자는 화면 이동 중 서버 데이터가 자주 바뀌기 때문에, React Query 캐시를 중심으로 서버 상태를 관리하고 UI 상태는 로컬 혹은 분리하는 방향을 선택했습니다.

실행

- 앱 소개 중심이던 웹을 실제 서비스 진입점으로 바꾸고, 탐색부터 결제까지 이어지는 흐름을 웹 안에서 완결할 수 있도록 직접 재구성했습니다.
- 로그인 상태와 서비스 데이터를 안정적으로 다룰 수 있는 상태 관리 구조를 정리해 주요 화면 경험을 일관되게 맞췄습니다.
- 웹에서 구현한 핵심 화면을 앱에서도 재사용할 수 있도록 연결 구조를 정리해 플랫폼별 기능 중복을 줄였습니다.

결과

- 웹 MAU가 8만에서 12만으로 증가했고, 전체 MAU는 25만+ 규모로 확장됐습니다.
- 모바일 웹, 데스크톱 웹, 앱 웹뷰 3개 접점의 사용자 흐름을 일관되게 맞춰 기능 확장 시 혼선을 줄였습니다.
- 웹에서 구현한 VOD 경험을 앱 상세/플레이어 화면에서도 재사용할 수 있게 만들어 동일 기능을 플랫폼별로 다시 구현하는 비용을 줄였습니다.
- 웹뷰 기반 재사용 구조를 정리해 앱 안에서도 웹 기능이 끊기지 않도록 연결했습니다.

Before

- 1 웹은 서비스 소개와 앱 다운로드 유도 중심의 랜딩 페이지
- 2 실제 탐색, 로그인, 결제는 모두 앱에서만 가능
- 3 웹 유입이 실제 사용 전환으로 이어지기 어려움
- 4 웹과 앱의 사용자 흐름이 분리되어 이용 경험이 자주 끊김

After

- 1 서비스 상세 페이지를 실제 서비스 진입점으로 확장
- 2 React Query 캐시 기반 서버 상태 관리 + 로컬 상태 분리
- 3 웹뷰 기반 재사용 구조로 앱 안에서도 웹 기능을 자연스럽게 연결
- 4 웹만으로도 탐색부터 결제까지 완성된 사용자 경험 구축

표준 개발 프로세스를 정립하고 AI 에이전트가 재현할 수 있는 개발 하네스를 구축

AI 개발 체계 · 프로젝트 산출물 완성 시간 기준 약 60% 개선

문제 인식

AI 에이전트를 활용하면 구현 속도는 빨라질 수 있지만, 개발 프로세스가 먼저 정립되어 있지 않으면 설계 없이 코드부터 작성하거나 백엔드, 프론트엔드, 모바일 산출물이 서로 다른 기준으로 만들어질 위험이 있었습니다. 특정 에이전트에서만 가능한 프롬프트가 아니라 어떤 AI 도구에서도 반복 가능한 개발 흐름이 필요했습니다.

분석

단순한 프롬프트 모음으로는 요구사항, 설계 산출물, 구현 코드, 테스트 케이스의 정합성을 유지하기 어렵다고 판단했습니다. 그래서 먼저 기획/SRS → WBS → 설계(DBML, Swagger, TCL) → 도메인별 구현 → 검증으로 이어지는 표준 개발 프로세스를 직접 정립하고, 에이전트는 이 프로세스를 실행하는 역할로 두는 구조를 설계했습니다.

실행

- 각 단계의 입력값, 산출물, 완료 조건을 정의해 AI 에이전트가 임의로 순서를 건너뛰지 않고 같은 흐름을 따르도록 했습니다.
- 설계 단계에서는 DBML, Swagger, TCL writer를 분리하고, 산출물 간 정합성을 검증하는 reviewer agent 흐름을 구성했습니다.
- 백엔드, 프론트엔드, 모바일 구현 단계는 domain expert sub-agent로 분리해 각 도메인의 개발 순서와 컨벤션을 독립적으로 준수하도록 설계했습니다.
- Claude, Cursor, Codex 등 도구별 차이는 어댑터로 흡수하고, 실제 프로세스와 규칙은 공통 원본을 참조하게 구성했습니다.
- 최종 검증 단계에서는 TCL 기반 Unit Test, E2E, 수동 체크리스트를 통해 요구사항과 구현 결과를 다시 연결했습니다.

결과

- 직접 정립한 표준 개발 프로세스를 AI 에이전트가 재현 가능한 형태로 시스템화했습니다.
- 역할 기반 sub-agent orchestration을 통해 어떤 AI 에이전트를 사용하더라도 동일한 단계, 산출물, 검증 기준에 따라 개발을 진행할 수 있는 구조를 만들었습니다.
- 설계 산출물, 구현 코드, 테스트 케이스가 하나의 흐름으로 연결되어 산출물 누락과 명세 불일치를 줄였습니다.
- 프로젝트 산출물 완성 시간 기준으로 개발 효율이 약 60% 개선되는 효과를 확인했습니다.

표준 개발 프로세스

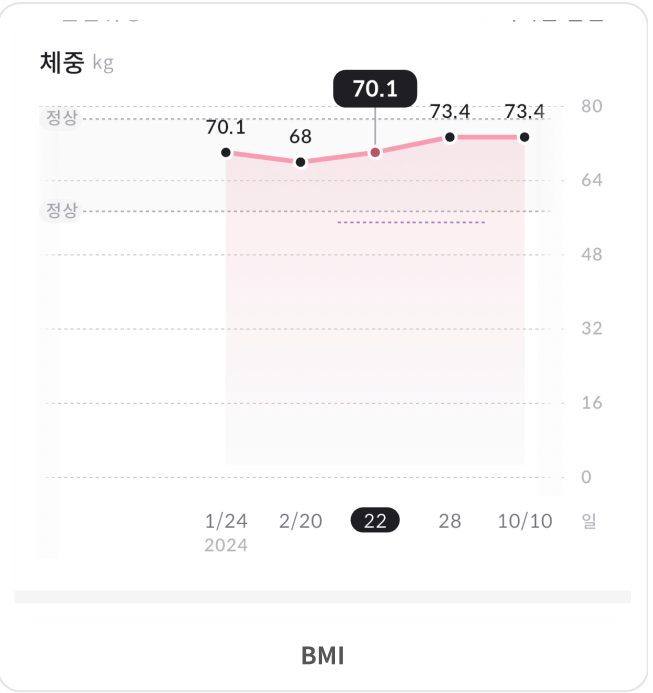
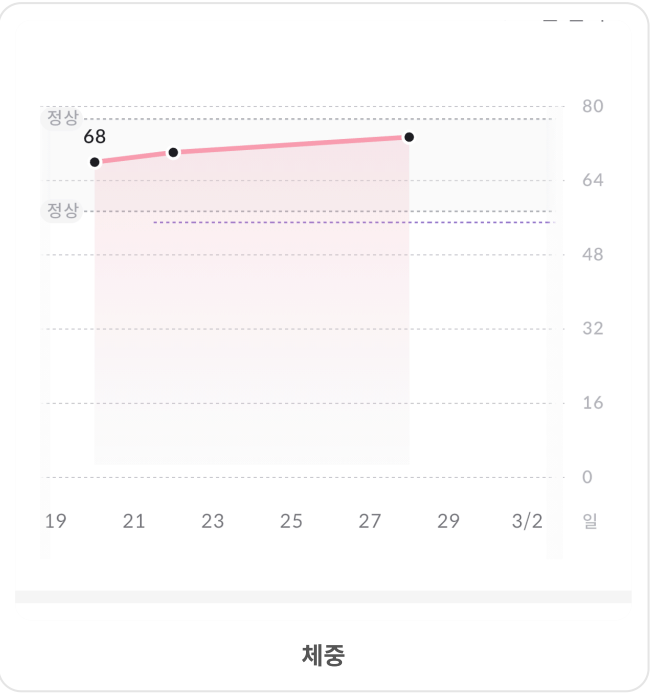
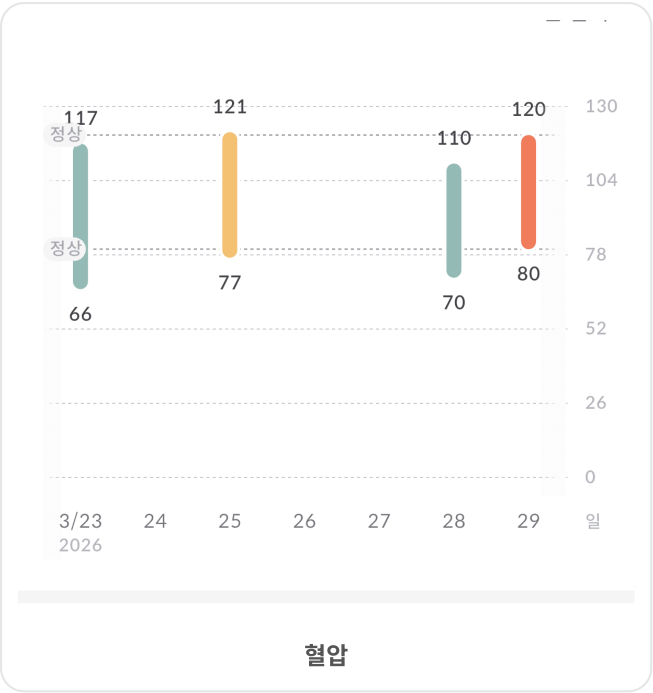
- 1 기획/SRS를 기준으로 요구사항과 범위 정리
- 2 WBS 작성 후 설계 산출물(DBML, Swagger, TCL) 생성
- 3 Backend / Frontend / Mobile 도메인별 구현 순서 적용
- 4 TCL 기반 Unit Test, E2E, 수동 체크리스트로 검증
- 5 요구사항 → 설계 → 구현 → 테스트가 이어지는 개발 흐름 완성

에이전트 실행 구조

- 1 메인 에이전트가 SRS, Swagger, TCL, 작업 범위를 확인
- 2 dev-chain 스킴이 단계별 입력값, 산출물, 완료 조건을 제공
- 3 Backend / Frontend / Mobile Expert sub-agent가 각 도메인 개발 순서를 준수
- 4 Reviewer agent가 설계 산출물 적합성과 누락 항목을 검증
- 5 어떤 AI 도구에서도 같은 개발 프로세스를 반복 실행

서로 다른 데이터 모델을 일관된 인터랙티브 차트 경험으로 통합

차트 시스템 설계 · 3개 지표 공통화



문제 인식

데이터 타입별로 화면을 따로 만들면 지표마다 계산 로직, 렌더링 방식, 인터랙션 처리가 분리되어 같은 앱 안에서도 차트 경험이 제각각이 될 위험이 컸습니다. 신규 지표가 추가될 때마다 화면을 다시 만드는 구조가 되면 개발 속도는 느려지고 유지보수 비용은 계속 증가할 수밖에 없었습니다.

분석

데이터 종류마다 화면을 따로 만드는 방식은 처음엔 빠를 수 있어도, 건강 데이터가 늘어날수록 차트 로직과 UX가 쉽게 분기될 수 있다고 봤습니다. 서버 DTO를 그대로 UI에 연결하면 데이터 타입이 늘어날 때마다 화면과 렌더링 로직이 함께 흔들릴 수 있어, DTO와 앱 모델을 분리하고 공통 차트 모델과 더 낮은 렌더링 레이어를 먼저 설계하는 쪽이 재사용성과 UX 일관성 모두에 유리하다고 판단했습니다.

실행

- 서버 DTO와 앱 모델 레이어를 분리하고, 팩토리 메소드 기반으로 데이터별 모델을 공통 차트 모델로 변환하는 구조를 직접 설계했습니다.
 - 혈압의 min/max 값과 혈당·체중의 single value처럼 형태가 다른 데이터도 같은 렌더링 구조에서 처리할 수 있도록 설계했습니다.
 - `CustomPainter` 와 `GestureDetector` 기반으로 차트 렌더링과 줌/스ワイ프 인터랙션을 직접 구현했습니다.
-

결과

- 혈압, 체중, BMI 3개 지표를 하나의 차트 시스템으로 일관되게 표현할 수 있게 됐습니다.
- lerp 기반 애니메이션과 제스처 처리로 요구사항에 맞는 차트 UX를 구현했습니다.
- 차트 로직의 재사용성과 확장성이 높아져 이후 건강 데이터 타입을 추가하기 쉬워졌습니다.

구현 흐름

- 1 혈압, 혈당, 체중 등 각기 다른 DTO 입력
- 2 DTO와 앱 모델 레이어 분리 + Mapper 규칙 정의
- 3 팩토리 메소드 기반 공통 차트 모델로 변환
- 4 CustomPainter 기반 렌더링
- 5 GestureDetector 기반 줌 / 스와이프 / 선택 인터랙션 적용
- 6 동일 구조 재사용으로 일관된 인터랙티브 차트 UX 구현